Hardware Acceleration of the LeNet-5-Architecture

Rechnerarchitekturen für Deep-Learning Anwendungen

Maxim Balajan and Markus Gerber

Friedrich-Alexander-Universität, Erlangen-Nürnberg

Matrikelnummer	23228101	22972477
StudOn-Kennung	hi46pubi	ba14acym
E-Mail	maxim.balajan@fau.de	markus.gerber@fau.de

1 Introduction [Shared]

With the increasing popularity and hype around Artificial Intelligence (AI), specifically around its subfield of Deep-Learning (DL), it is no wonder that demands for such models are steadily increasing. One of the essential disciplines of DL are Convolutional Neural Networks (CNNs), mainly used for image recognition/classification in many different areas, such as medical imaging or autonomous driving.

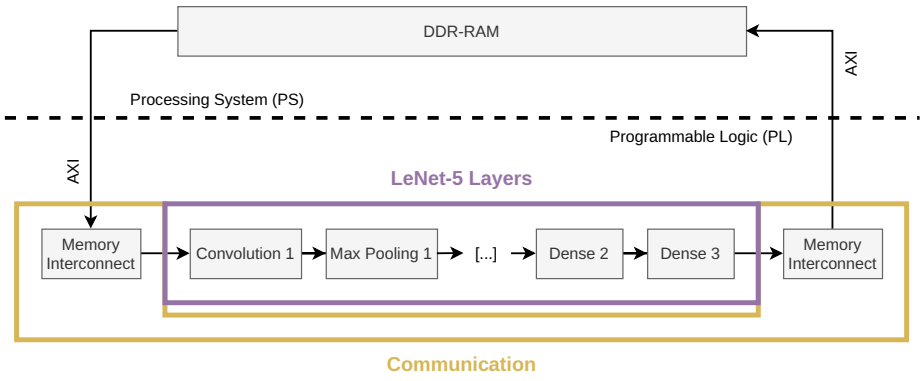


Fig. 1: Generic Accelerator Design [13]

Primarily in use cases such as autonomous/assisted driving, image inference with low latency is of utmost importance. There are many possibilities to achieve this, either by implementing standard algorithmic optimizations or utilizing specialized hardware like GPUs or TPUs. This report, however, proposes an improvement by utilizing a customized hardware accelerator mapped onto a Field-Programmable Gate Array (FPGA).

For this work, the accelerator is implemented on the Ultra96-v2 [8], a MPSoC consisting of a Processing System (PS) and a Programmable Logic (PL). Figure 1 shows the proposed idea, which is highly inspired by the work of authors in [13] where the image data is streamed from the PS into an accelerator placed on the PL of the FPGA. The resulting label of the image is then streamed back to the PS.

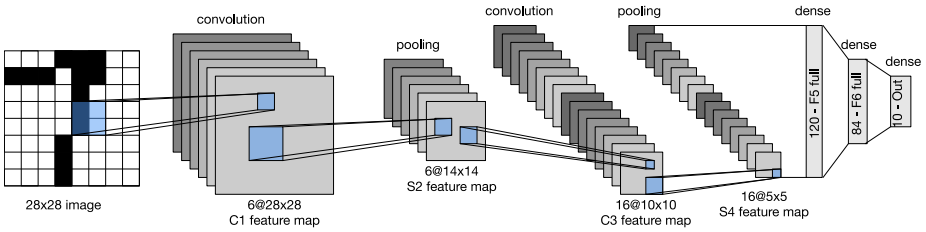


Fig. 2: Overview of LeNet-5 Architecture [3,11]

Because this work should only serve as a prototype, the CNN under consideration will be the LeNet-5 Architecture [11], which classifies images of the MNIST dataset. This dataset comprises a few thousand handwritten digits represented by $(28 \times 28)-$pixels. The corresponding visual representation of the model can be seen in Figure 2. The accelerator is also built around the restriction that $batch_size = 1$, meaning only one image can be inferred at a time. This restriction was imposed on the project to reduce its complexity further.

2 Prerequisites [Shared]

To understand certain nuances of the accelerator design, a few prerequisites must be considered first. Because the accelerator will only focus on the inference, the training has to be achieved through other means. This training phase, as well as the process of *Quantization*, are further elaborated in Section 2.1 in greater detail.
Section 2.2 discusses the software implementation of the LeNet-5 model, which serves as a reference for verifying, debugging, and benchmarking the hardware-accelerated model.

2.1 Training and Quantization [Maxim]

The training phase was achieved via a Python script using PyTorch [7], or, to be more precise, the quantization framework of Xilinx called Brevitas [9].
Normally, the weights and inputs inside neural networks are stored as Single-Precision Floating-Point-Values for the training and inference phase. This way, of course, is fairly resource intensive considering that for every computation, a Floating-Point-Unit (FPU) is necessary and that 32-bit values need to be stored for every intermediate register.
Quantization aims to map these Floating-Point-Values onto Fixed-Point-Values with smaller bit widths. In our case, the quantized values have a width of 8 bits. The computations can also be achieved with standard integer arithmetic, significantly reducing resource overhead. There are a few things to take care of, for example, the clipping and scaling of intermediate results, as explained in this article [1], but these extra steps are still far cheaper than utilizing FPUs.
The quantized model and the Quantized-Aware-Training were all achieved via Brevitas over a training duration of 100 Epochs using the training data of the MNIST Dataset provided by PyTorch. The model achieves an accuracy of $\approx$ 98.91% on over $10,000$ test images.
The model was then exported using the ONNX format [5], and the images and their corresponding labels were exported as NumPy-Arrays to reuse these for the accelerator and the software implementation.

2.2 Software Implementation [Markus]

In this section, the software reference implementation is introduced. First, Section 2.2.1 discusses the main reasons for its importance and lays out the main design goals. Then, Section 2.2.2 examines the choice of bit widths and the effect of clipping values on the inference accuracy and chooses a good tradeoff to reduce hardware costs. Finally, Section 2.2.3 explains how the software implementation handles the model and image data. This section includes how the data is parsed, preprocessed, and transferred to the target system.

2.2.1 Reasoning [Markus]

Although the software implementation might not seem to be directly related to the hardware-accelerated model, it is crucial for the development and evaluation process. There exist five main reasons for this:

1. **Configuration:** Choosing a minimal architecture and constraining it further can drastically reduce the hardware required for the LeNet-5 hardware accelerator. In this case, a good bit width for both the main data and intermediate accumulators is proactively searched for. Additionally, to find a good tradeoff between bit width and whether clipping is used, the effect of clipping data is to be evaluated for each bit width.
2. **Verification:** Before developing hardware to fit a specific problem, verifying the correctness of architectures using a reference model that mimics all operations to be implemented is a huge advantage. This is especially true for quantizing the LeNet-5 model, as the quantization process can quickly introduce errors.
3. **Reference:** While implementing a target architecture, having a central location to look up important details is beneficial. This is also true for the quantized LeNet-5 architecture targeted, as specific aspects differ from existing online resources.
4. **Debugging:** As developing hardware can easily result in really hard-to-find bugs (e.g., because of timing issues), having an easy way to debug single components step-by-step can drastically reduce the time needed for debugging. For this reason, the software-reference model is used to compare its intermediate results to the ones of the LeNet-5 hardware accelerator. Existing libraries such as PyTorch or TensorFlow are not suited for this purpose as their architecture might be slightly different, and might not provide sufficient access to these intermediate results and detailed debugging information.
5. **Benchmarking:** To evaluate the hardware accelerator, its performance is compared to other implementations used for inferring CNNs. Although existing libraries such as PyTorch or TensorFlow are well suited for inference, they are the product of many years of development (i.e., they are highly modular and highly optimized). Exclusively using such libraries as a baseline does not offer a realistic comparison. Instead, the custom and easy-to-port software-reference model is employed as a fair competitor.

The software-reference model is proactively implemented as a flexible, modular, and hardware-agnostic C++ library for these reasons. To provide a realistic baseline for evaluation, the reference implementation also aims to be relatively efficient, avoiding dynamic memory allocation during runtime, for example.

2.2.2 Bit Widths & Clipping [Markus]

To be able to test out different data types and simulate their behavior, the software-reference model is designed to be highly flexible. This flexibility is implemented using C++'s template system, which allows for easily swapping out data types statically without any additional runtime penalties. An example of this can be seen in Listing 1, where the main and accumulator types are chosen and used afterward to parse the pre-trained LeNet-5 model and run the inference. As native data types (e.g., `int8_t` or `int32_t`) cannot provide advanced functionality such as clipping values or simulating 3-byte integers, custom data types can be implemented. An example would be the `ClippedInt8` type[1], which is used in Listing 1 as the main data type. To further simplify development, these custom data types overload the most important operators (e.g., `+`, `-`, `*`, `/`) and therefore allow for transparent access to the underlying data.

Listing 1 : Choosing & using the generic target types on a high level

```
1  // choose target types (e.g. int8_t, Int24 (custom), ClippedInt8 (custom), ...)
2  using MainType = ClippedInt8;
3  using AccumulatorType = ClippedInt24;
4
5  // parse & preprocess layers
6  auto layers = parse_layers<MainType, AccumulatorType>("LeNet-5.onnx");
7
8  // run inference
9  const Matrix<MainType>* curr_input = &input;
10 for (auto& layer : layers) {
11  curr_input = &layer.calc(*curr_input);
12 }
```

Inference is run for all 10,000 test images provided by the MNIST dataset to evaluate the accuracy for different configurations. Figure 3 shows the inference accuracy for a *default* implementation, and one that *clips* all values to their corresponding minimum and maximum. Figure 3 makes it clear that clipping values increases inference accuracy significantly. Therefore, it seems beneficial to employ clipping for all further computations in favor of smaller bit widths. Furthermore, based on the findings in Figure 3, the main data is chosen as 8 bits in size, while accumulators are to be composed of 24 bits. Further reducing the accumulator bit width to 16 bits seems possible.

[1] Because of previous versions of the reference implementation, a `ClippedInt8` wrapper is implemented by the `FPInt8<0>` type.

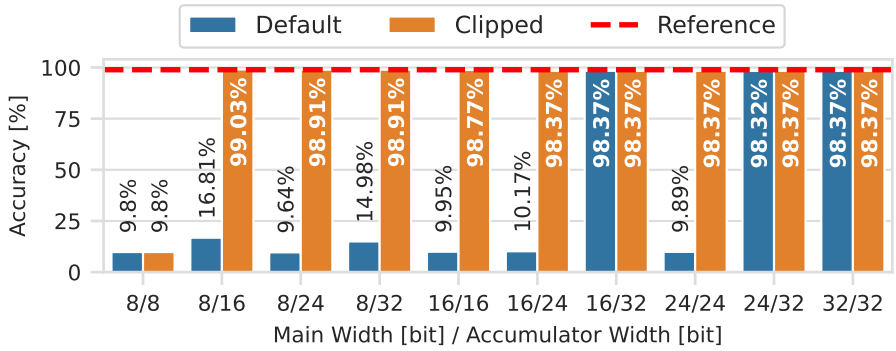


Fig. 3: Inference Accuracy for different Data-Type Configurations

2.2.3 Handling Model & Image Data [Markus]

As mentioned in Section 2.1, the quantized LeNet-5 model is saved in the ONNX format after training it. However, before using it, parsing and preprocessing are necessary. For parsing the ONNX format, Google's Protocol Buffers (protobuf) [6] library is utilized, which is the base for ONNX. Afterward, each layer is mapped to its internal representation[2]. During this phase, the weights and biases are also loaded and converted into the correct format. For example, weights that are stored as `float`s will be translated to the correct data type of the target (e.g., `int32_t`, `int8_t`).

For the test images, this process is similar. First, the images are loaded from NumPy's data format using the open-source `libnpy` [4] library. Then, images are preprocessed, that is to say, they are zero-padded, scaled as specified by the quantization parameters, and finally converted to the target data type.

As the software-reference model is designed to be hardware-agnostic, while some of its parsing dependencies are not, model and image data are parsed and preprocessed on a host system. Afterward, the data is serialized and transferred to the target system, after which it is deserialized for further use.

[2] The quantized model exported by Brevitas uses dedicated *Quant* layers that contain scaling and offset information. The implemented reference simulation, however, integrates these into the corresponding convolutional and dense layers.

3 Architecture Design [Maxim]

Before we take a deep dive into the different components that make up the accelerator and the parts necessary for its software integration, let us first look at the general design of the accelerator and what general components it consists of. Figure 4 illustrates the design, consisting of the first two convolutional and pooling phases in the beginning and the three dense layer phases at the end. The following sections will go over the individual steps in more detail. An in-depth look into the separate components can be found in Section 4.

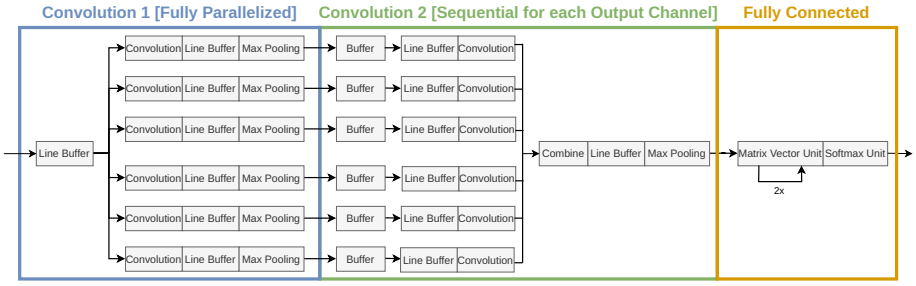


Fig. 4: Proposed Accelerator Design for the LeNet-5 Architecture (*lenet_accel.vhd*)

3.1 First Convolutional Layer [Maxim]

As seen in Figure 1, the image data is streamed directly from the PS one pixel after the other over the memory interconnect. This data stream is fed into the accelerator and the first convolutional phase. From Figure 2, it is apparent that six output channels are computed for one input channel represented by one source image. Because of this, only six convolutions need to be computed in total, which can easily be fully parallelized. The *conv_par* unit implements this parallelization of convolutions.

The input pixel values are first passed into a Kernel-Slider consisting of Linebuffers, which retrieves all essential kernel windows for the first convolution. These kernels are then passed into the different output channels, where the convolution is calculated via MAC operations together with the individual channel weights. After that, the new image data is streamed to a different Kernel-Slider, which gets all kernel windows necessary for the MaxPooling-Unit. A more detailed explanation of the various components featured in this convolutional phase can be found in Section 4.1.

This approach retrieves all pixel data of all six output channels simultaneously and passes everything to the second convolutional layer for further computation.

3.2 Second Convolutional Layer [Markus]

As the data for all six output channels of the first convolutional layer is computed step-by-step, it is directly streamed into the second convolutional layer. This second layer is significantly more complex than the first, as it must calculate 16 output channels based on six input channels. Fully parallelizing this operation would require a substantial amount of hardware. Additionally, for such parallelization to be beneficial, the resulting data stream would also have to be processed in parallel. This approach is not feasible for the targeted platform.
Instead, the second convolutional phase is designed to compute each output channel sequentially. The *conv_seq* unit implements this and works as follows.

1. One byte after another arrives at the second convolutional layer for each input channel. As shown in Figure 4, each input channel forwards its corresponding data to a buffer that stores the received data and then forwards it to the processing part of the second convolutional layer. Similar to Section 3.1, this consists of a Kernel-Slider (with Linebuffers) and the component calculating the convolutional operation, with the weights for the first output channel. However, unlike Section 3.1, combining all six input channels to compute the first output channel is necessary. This combining phase is accomplished by adding all six intermediate results and passing them through another Linebuffer that forwards the correct kernels to the final MaxPooling-Unit.
2. The entire output channel is computed step-by-step in this manner, and the resulting data stream is passed to the next fully connected layer.
3. Once an output channel is fully computed, the *conv_seq* unit resets all its internal components, except for its data buffers, which store one complete input channel each. It also switches the convolution weights to those for the next output channel.
4. The *conv_seq* unit then instructs its data buffers to stream each stored input channel to the processing part of the second convolutional layer.
5. Steps 2, 3, and 4 are repeated for all remaining output channels.
6. After 16 total iterations, all 16 output channels were fully computed and passed to the first fully connected layer.

Although this approach computes all 16 output channels of the second convolutional layer sequentially with an acceptable amount of hardware, there is still room for improvement. The current implementation of the *conv_seq* unit waits until the output channel is finished computing before switching to the subsequent output channel. Waiting, of course, is not ideal, as such an implementation drains the whole pipeline between switching output channels. The additional latency this introduces for each channel switch is exactly the latency it takes for one byte to pass through the whole pipeline.
One potential solution is to dynamically pipeline the reset signal and switch output channels. However, since Linebuffers cannot be partially reset, this approach would offer minimal latency improvement.
An alternative solution involves using two multiplexed Kernel-Sliders, streaming

the corresponding input channels delayed, but still simultaneously. This alternative procedure allows the Kernel-Slider of the previous output channel to complete its data output. In contrast, the Kernel-Slider for the next output channel begins receiving and buffering new data. Ideally, this approach would eliminate additional latency when switching between output channels while only slightly increasing hardware requirements. This increase should not exhaust the remaining resources, as detailed in Section 5.1. However, further evaluation is needed to determine this approach's actual hardware requirements and performance gains.

3.3 Fully Connected Layers [Maxim]

After having computed both convolutional layers, all three fully connected layers must be computed one after the other, as seen in Figure 2. This computation is being done sequentially over three iterations.

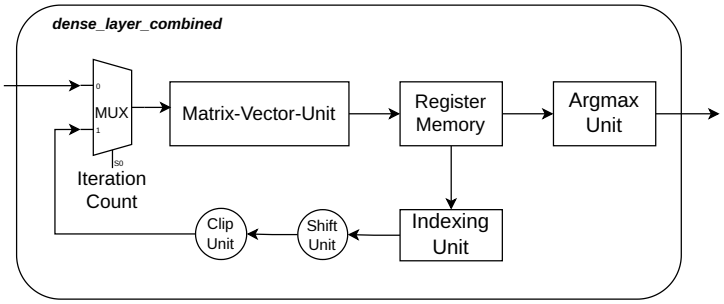


Fig. 5: Simplified Structure of the Fully-Connected Layers (*dense_layer_combined.vhd*)

Figure 5 illustrates the data flow present inside the *dense_layer_combined* unit with simplified control logic. In the first iteration, the data is streamed from the second convolutional layer directly into the Matrix-Vector-Unit, which, as the name suggests, computes the product of a weight matrix together with an input vector. As apparent from the figure and later from the more detailed overview in Section 4.2.1, this unit is only instantiated once as it is fairly resource intensive. For this reason, the unit is being reused for all three iterations.
After the multiplication has been computed, the result vector is stored inside registers for later use in the subsequent iteration phases. It is important to note that the registers store the result with 24 bits without shifting and clipping them beforehand (compare Section 2.1). Otherwise, multiple shifting and clipping units would be necessary, leading to a much bigger resource overhead.

Rather than having multiple shifting and clipping units, only one each is instantiated. They directly follow an indexing unit, which has the purpose of retrieving one value at a time from the register memory and stream it to the Matrix-Vector-Unit.
After computing all three iterations, the temporary result vector is shifted, clipped, and then passed to the Argmax-Unit to determine the actual label of the input image. For a more detailed explanation of the Argmax-Unit, the Matrix-Vector-Unit, and its memory interconnect, see Section 4.2.

4 Implementation [Shared]

4.1 Components for the Convolutional-Layers [Maxim]

4.1.1 Kernel-Slider (Linebuffer) [Maxim]

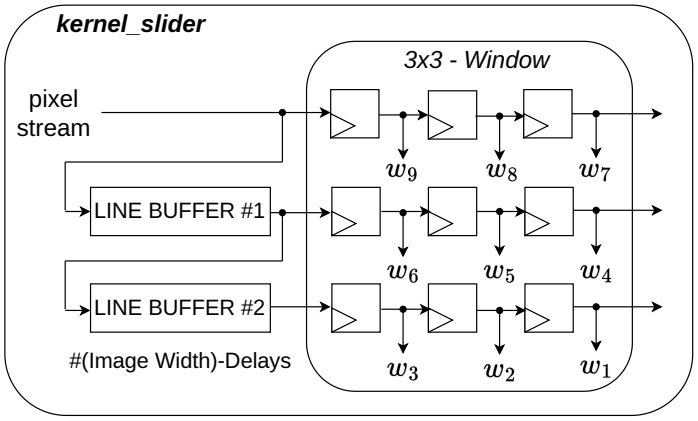


Fig. 6: Parallelized (3×3)-Kernel-Slider using Linebuffers [10] (*kernel_slider.vhd*)

As previously described in both convolutional phases, Kernel-Sliders, often also referred to as Window-Buffers, are used to retrieve all valid kernel windows of an input stream. Figure 6 illustrates an example of the circuit for $kernel_size = 3$. The entire image data is streamed into the Kernel-Slider, where the data is delayed via $kernel_height - 1$ many Linebuffers. These buffers are just RAM-based FIFOs with $depth = image_width$ that only output their entries if the buffer is full. Over these delays, the desired rows of the kernels are achieved.
The Kernel-Slider was written in a generic way so that it can be instantiated for both the convolutional and the pooling phases. The $kernel_size$ can also be statically changed depending on which $kernel_size$ is necessary. With that, the desired $kernel_size = 5$ for the convolution and $kernel_size = 2$ for the pooling can easily be set.
Although the general principle of this module is relatively easy to implement, controlling it comes with its own challenges. Figure 7 illustrates one of these problems. During its strides, the kernel will eventually walk Out-of-Bounds at the end of each row for $kernel_width - 1$ many cycles. Because of that, the kernel needs to be flagged as invalid for these cycles.
Another problem lies in the difference between the convolution and the pooling regarding their $stride_length$. For the pooling phase, $stride_length = 2$ is always the case. This difference means that every other kernel in the x-direction and every other row in the y-direction must be flagged as invalid. For the convolutional phases, this problem does not apply because of their $stride_length = 1$.

These separate behaviors for controlling the Kernel-Sliders were realized in both the *convolution_slider* unit and the *pooling_slider* unit, respectively.

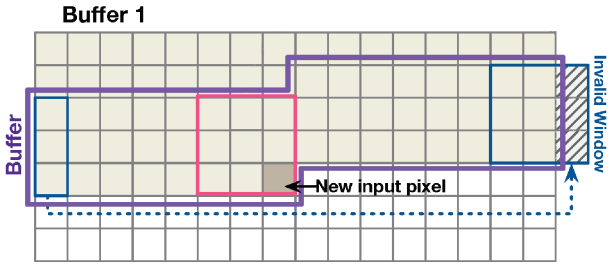


Fig. 7: Occurrence of invalid Windows in a Kernel-Slider [12]

4.1.2 Kernel-Computation-Unit [Maxim]

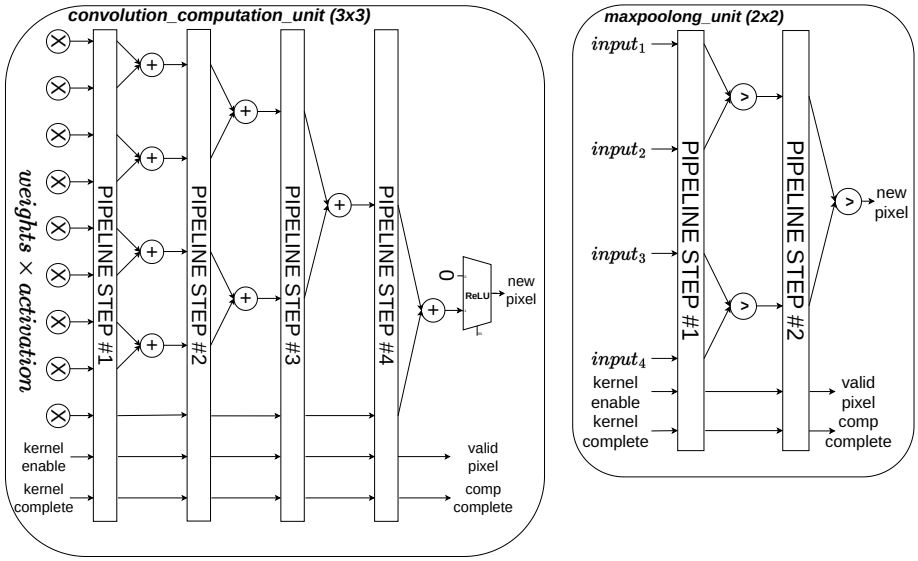


Fig. 8: Both main Computation Units inside *kernel_computation_unit.vhd*

The *kernel_computation_unit* is a fairly interesting unit because, unlike the other modules inside the accelerator, this unit serves multiple purposes. Figure 8 depicts both major versions present inside this unit. As apparent from the figure, this unit can either be instantiated as a *convolution_comp_unit*, which

computes the summed-up products of the weights and their activations, or as a *maxpooling_unit*, which, as the name suggests, retrieves the maximum of all input values.
This design decision was made because of the structure that resulted from implementing all present pipeline steps. The entire design is implemented completely generically, meaning that when the input size is changed before the instantiation of the module, the entire pipeline structure is built automatically around it to accommodate these changes. This pipeline structure was achieved by implementing a generic pipeline step in the *kernel_computation_step* unit that is instantiated multiple times to achieve the desired structure.
The convolutional version of this unit can also be customized to include or exclude specific pipeline steps at the beginning or the end. In the case of the version seen in Figure 8, the multiplication step at the beginning could be disabled. Also, the ReLU step at the end can be disabled, or if the ReLU step is present, a shifting pipeline step could be added, which shifts the temporary results accordingly (compare Section 2.1). These customizations are utilized, for example, in the *Combine* step in the second convolutional phase.
Another important thing to mention is that all adders present inside the convolutional version of this unit can be either instantiated as normal adders or as *clip_adders*. These adders with clipping functionality can be seen in Figure 9 and serve the purpose of preventing over- and underflows. However, compared to all other parts inside the Kernel-Computation-Unit, these adders are purely combinatorial and not pipelined. Although pipelining these would not be too challenging inside the convolutional phases, this would pose a big problem for the Matrix-Vector-Unit utilized inside the fully connected layers because of the looped structure inside the MACs (compare Section 4.2.1).
Although this unit is amazing to use, these major functionalities, in the form of a convolutional and a pooling unit, should have been separated because the design became increasingly unclear as the features had to increase. However, this unit worked incredibly well for our use cases, which is why it was decided against refactoring it.

4.1.3 Convolutional-Pooling-Pipeline [Maxim]

With both the Kernel-Slider and both versions of the Kernel-Computation-Unit out of the way, the pipeline used in the first and second convolutional phases can almost be constructed. The only thing missing is how the weight memory for the convolution is realized.
Figure 10 shows how the memory is retrieved in the convolutional phases. All important data is stored inside a Single-Port Block Random-Access Memory (BRAM). During the initialization phase of the Kernel-Sliders, all 25 weight values are stored inside intermediate registers to be used whenever needed.
This way of retrieving the memory is a fairly intuitive approach for acquiring the necessary data; what is trickier is how the important data is written to the BRAM in the first place. The process for obtaining the memory, together with why the base address is necessary, will be elaborated in Section 4.3.2.

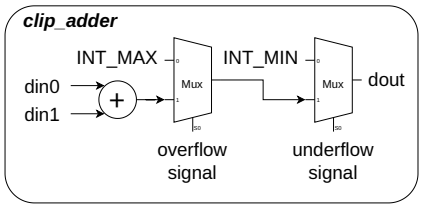


Fig. 9: Clip adder used for Additions (*clip_adder.vhd*)

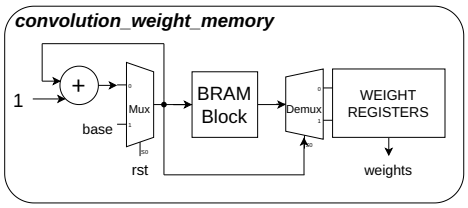


Fig. 10: Memory of the Convolution (*conv_mem_unit.vhd*)

With that, all important components are ready to be implemented inside the Convolutional-Pooling-Pipeline that was already hinted at in Figure 4.
Figure 11 showcases the resulting structure of such a pipeline. It is important to note that the first Kernel-Slider (Linebuffer) necessary for the convolutional part of the pipeline is not inside the module. This design decision was made because, as seen in the accelerator structure, the first convolutional phase uses one singular Linebuffer for all six output channels, which is why the buffer lies outside the module.

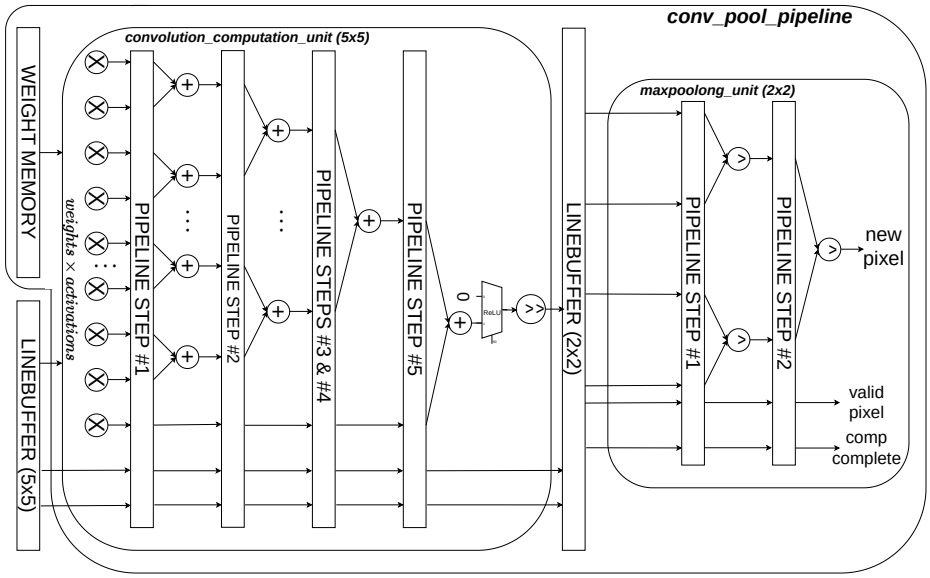


Fig. 11: Convolutional-Pooling-Pipeline used in both Convolutional Phases (*conv_pool_pipeline.vhd*)

4.2 Components for the Fully-Connected-Layers [Shared]

4.2.1 Matrix-Vector-Unit [Maxim]

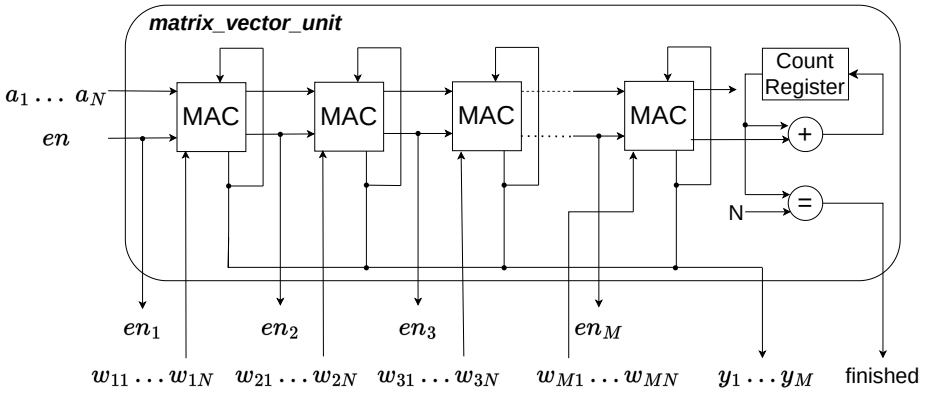


Fig. 12: Unit for computing the Product of a Matrix-Vector-Multiplication (*matrix_vector_unit.vhd*)

In order to compute fully connected layers, a unit for computing Matrix-Matrix-Multiplications and Matrix-Vector-Multiplications is necessary. Luckily, because of the restriction mentioned in Section 1 ($batch_size = 1$), only a unit capable of Matrix-Vector-Multiplications is needed.
First, an approach was tried that did not consider this restriction. This approach resulted in an unrealistic circuit that would not scale for this project's use case.

$$\begin{bmatrix} w_{11} & w_{12} & w_{13} & \cdots & w_{1N} \\ w_{21} & w_{22} & w_{23} & \cdots & w_{2N} \\ w_{31} & w_{32} & w_{33} & \cdots & w_{3N} \\ \vdots & \ddots & \ddots & \ddots & \vdots \\ w_{M1} & w_{M2} & w_{M3} & \cdots & w_{MN} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ \vdots \\ a_N \end{bmatrix} = \begin{bmatrix} y_1 \\ y_2 \\ y_3 \\ \vdots \\ y_M \end{bmatrix}$$

Fig. 13: Arithmetic Operation realized by the Matrix-Vector-Unit

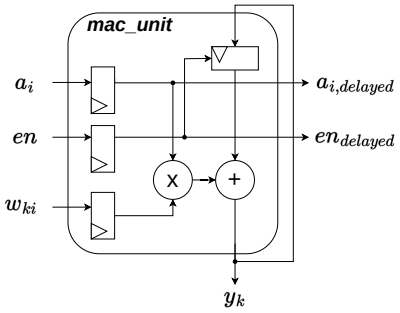


Fig. 14: Unit for computing MAC operations (*matrix_vector_mac_unit.vhd*)

Figure 12 illustrates the design that was ultimately settled on, which takes the previously described restriction into account and computes the product of a matrix and a vector as seen in Figure 13. The unit is a systolic array that consists of chained MAC units as depicted in Figure 14. One singular MAC-Unit computes the product of the input vector with one row of the weight matrix. Because a singular input value is passed down from MAC to MAC, the weights must be delayed accordingly to time everything correctly. For this purpose, the individual enable signals are passed onto the memory layer necessary for this unit, which will be elaborated in more detail in Section 4.2.2.
As mentioned, this unit is only instantiated once for all three fully connected layers. From Figure 2, it is apparent that the first layer has a weight matrix with the dimension (120×400), which is why the instantiated module needs to have 120 MAC-Units. If a different design approach had been chosen where the Matrix-Vector-Unit would have been instantiated three times, roughly doubled the number of MAC-Units would have been necessary, increasing the hardware resources significantly. Because of this, the approach presented in Section 3.3 was used.
This repurposing of the Matrix-Vector-Unit also means that the unit needs to handle smaller vector sizes in order to use it for all three iterations. This is achieved by having the *weight_width* and the *weight_height* registers, which are loaded during the reset. These signals define which enable signal to increment the Count Register with to know when the unit finished its computation. Of course, this adds more complexity in the form of dynamic multiplexing, but it is still far cheaper than having three separate Matrix-Vector-Units.

4.2.2 Matrix-Vector-Memory-Interconnect [Maxim]

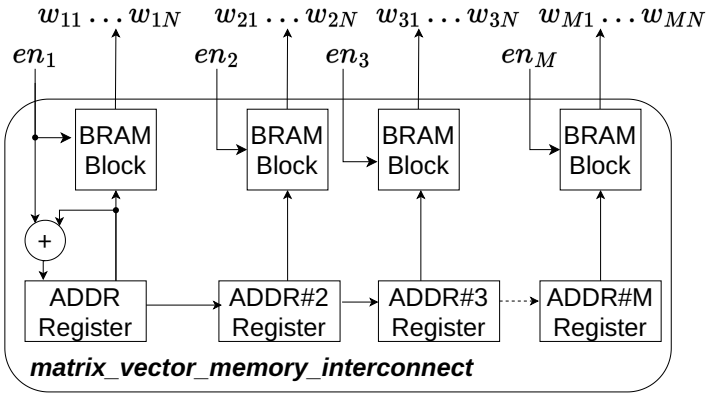


Fig. 15: Unit for retrieving the Weights of the Matrix-Vector-Multiplication (*matrix_vector_mem_interconnect.vhd*)

As mentioned in the previous section, a delay-based weight memory interconnect is needed for a systolic array. Figure 15 illustrates the necessary memory layer of the Matrix-Vector-Unit.
Because every MAC is delayed one cycle more than the previous one, the corresponding weight rows for a MAC-Unit, stored in BRAM tiles in our case, must be delayed similarly. This delay process is achieved via a shift register of the addresses, where the original address register is accumulated with the first enable signal provided by the Matrix-Vector-Unit every time a new valid value enters the pipeline. Over the subsequent iterations, this address is shifted through the address pipeline to achieve the desired delay.
This addressing could also be achieved by having one adder per BRAM tile, but this approach would come with a much higher resource utilization while delivering no extra benefits.
Another important aspect to talk about is how the memory is structured exactly. One single Dense-Layer achieves this by having one weight row per BRAM tile. Nevertheless, as previously described, this unit needs to be reused three times, making this endeavor more challenging. These challenges and their corresponding solutions will be elaborated further in Section 4.3.2.
Now that both important submodules necessary for calculating matrix-vector products have been covered, they can be connected to receive the primary unit of our Dense-Layers, which is seen in Figure 16.

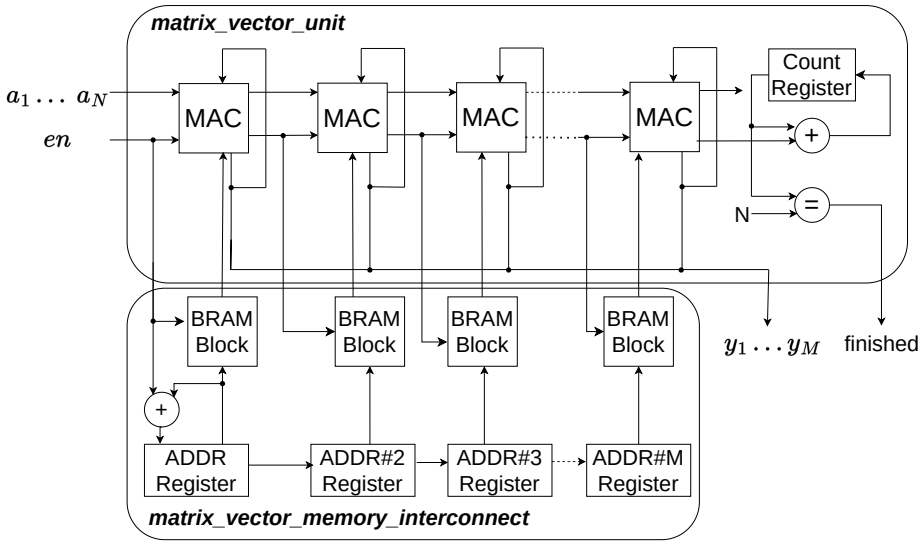


Fig. 16: Top-Level Unit for computing Matrix-Vector products (*dense_layer.vhd*)

4.2.3 Argmax-Unit [Markus]

After computing all three fully connected layers and applying shifting and clipping to all values, the resulting vector contains 10 values representing the probabilities of the input image corresponding to each digit. As the LeNet-5 accelerator should only output the index of the most probable digit, the argmax operation is used to determine the label of the input image. Although argmax [2] is usually defined to yield all indices of the maximum value in the input vector, only the smallest index is considered for this implementation. For vectors containing 10 elements, this is formally defined as:

$$\text{argmax}(x) := \min\{i \in \{0, ..., 9\} \mid \forall j \in \{0, \ldots, 9\} : x_i \leq x_j\}, \quad x \in \mathbb{R}^{10}$$

For example, for vector $v = \begin{pmatrix}1 & 27 & -20 & 2 & 43 & 10 & 7 & 2 & -15 & 10\end{pmatrix}^T$, argmax returns 4.

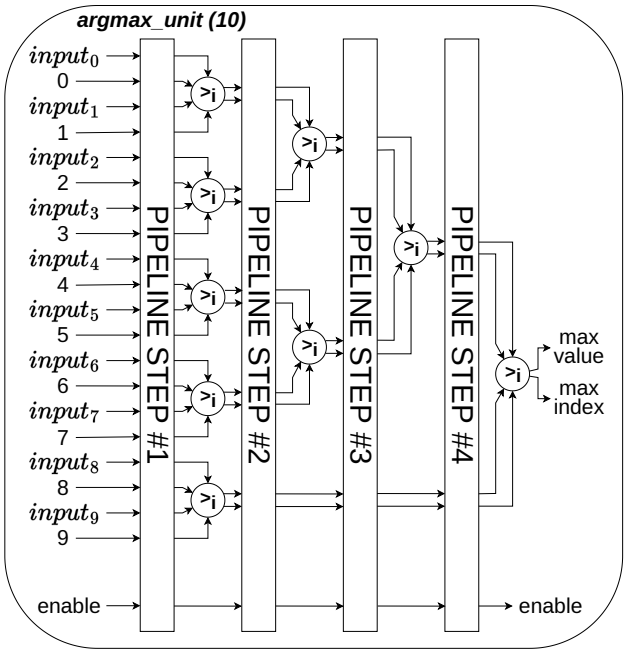


Fig. 17: Unit for computing the Argmax Operation ($argmax_unit.vhd$)

To implement this operation, an approach similar to the Kernel-Computation-Unit (compare Section 4.1.2) was chosen. Therefore, the computation is pipelined, and the unit is designed to be generic to support any vector size.
Figure 17 provides an overview of the resulting unit for a vector of size 10. In this example, the unit receives an enable signal and 10 scalar values as inputs, which

are processed in a pipelined manner. Because the argmax operation requires both a maximum value and its corresponding index for computing the index of the largest value internally, the unit generates a constant signal representing the index of the maximum value internally. This signal is also pipelined.
As shown in Figure 17, for each pipeline step, the number of values to be processed is reduced from n to $\lceil \frac{n}{2} \rceil$ until only one value remains. To reduce two values into one, a custom comparator ($<_i$) compares two values and outputs the larger value along with its index. This approach completes in 4 pipeline steps for a vector of size 10, which is optimal for binary operators.

4.3 Packages [Maxim]

4.3.1 Libmatrix & Libutil [Maxim]

Although VHDL is mostly used for its hardware description feature sets, it also comes with many imperative programming paradigms that make the simulation and preprocessing of units before synthesis more easily automated. These processing functionalities were encapsulated inside various packages that are reused in most of the units present in the design.
The *libmatrix* package does what the name suggests and provides various multidimensional *std_logic_vector*-types that are used to represent single- or two-dimensional representations of the images streamed through the accelerator. The package also has various features to transpose matrices or concatenate single-dimensional array types. These global type definitions were not necessary but very convenient when creating the interfaces of several submodules.
On the other hand, the *libutil* package is mostly used to make definitions of generics more flexible and to provide many convenient functionalities used for preprocessing inside the accelerator. For one, it provides string and integer lists as global types, mostly used inside the fully connected layers. With this approach, the architecture of this part could easily be extended to feature more or less iteration phases depending on the needs of the top-level module. For example, the various functions present are used in the Argmax-Unit to instantiate the labels necessary for image classification.
There are many more instances where both of these packages were utilized. These are just a few reasons why this form of abstraction was used in the design.

4.3.2 Libmemory [Maxim]

The *libmemory* package is probably the most integral for our design because it provides us with many impure functions for correctly initializing the BRAM memory found in our design. In this package, various functions for reading the raw weight layers are provided by our training script (compare Section 2.1), which enables us always to choose the correct initialization routine needed for the separate layers.

The first Convolutional-Layer computes all output channels for a given input channel and has one BRAM tile per output channel for its weight memory (compare Section 3.1). Because of this, one BRAM block must store all input channels for a given output channel. For our use case, we store one (5×5) weight window in our separate blocks because we only feature one input channel in the first place. This behavior is implemented inside the *read_output_channel* function.

Unfortunately, the second convolutional layer is trickier than that. In this phase, all output channels are computed sequentially one after the other, meaning that for one given output channel, the convolution for all input channels is computed (compare Section 3.2). Because of this, we have one BRAM tile per input channel, which results in 16 (5×5) weight windows that must be stored by a single BRAM block for the second convolutional phase. For this reason, the convolution memory provides a *base_addr* register with which the address offset of the unit can be set during a reset. This initialization routine is realized in the *read_input_channel* function.

Finally, the three dense layers that conclude the design are the only ones missing. In this phase, one MAC-Unit computes the multiplication of one weight row with its corresponding activation vector (compare Section 3.3). For this reason, the corresponding BRAM block needs to store such a weight row. Nevertheless, because the Dense-Layer is reused for all three iterations, the memory not only consists of one weight row but can consist of up to three weight rows stored in series. It is important to stress that not all weights for a MAC store up to three weight rows because the subsequent iterations of the Fully-Connected-Layers have fewer rows, resulting in a few MACs only needing one or two weight rows. This functionality is encapsulated inside the *read_weight_row* method.

There are also a few other functionalities, such as the *read_scale_values*, which reads the shifting values needed at the end of every layer, or the $init_file$, which is mainly used for automated testing inside the test benches. However, the functionalities listed above are the most essential ones present.

4.4 Integration [Markus]

This section outlines how the LeNet-5 hardware accelerator is integrated into the Ultra96-v2 board so that it is usable by the PS. First, Section 4.4.1 discusses the design decisions to achieve high efficiency and low latency. Section 4.4.2 then gives an overview of how exactly the PS and the hardware accelerator communicate with each other. Finally, Section 4.4.3 shows in detail how the memory interconnect on the PL for managing the hardware accelerator is designed and implemented.

4.4.1 Considerations [Markus]

As pointed out by Section 1, efficiency and low latency are of utmost importance for the accelerator. To achieve this, this section first considers different design architectures.

Programmed Input/Output (PIO) vs. Direct Memory Access (DMA). For transferring data between the PS and the PL, two main approaches are possible, namely PIO and DMA. In the first one, the Central Processing Unit (CPU) actively transfers all the data, while in the latter, the CPU only initiates the data transfer, and the corresponding DMA controller takes care of the rest. As high efficiency is to be achieved and CPU time is a valuable resource for high-performant applications, *DMA* is chosen. This approach especially allows the CPU to run another task (e.g., retrieve the next image) while data is transferred.

Polling vs. Interrupts. After inference is finished and the corresponding image label is stored in Random-Access Memory (RAM), applications require a fast way to be notified and handle the resulting image label. One approach is for the PS to repeatedly check the status of the computation (*polling*) until the computation is completed. This pooling can either be done permanently or, as the hardware accelerator has a deterministic latency, using hardware timers to check the computation status only after a certain amount of time. Although the latter seems to be a good compromise, it is not optimal, as changes to the hardware accelerator can lead to a change in latency. In either case, computing time is too easily wasted.
An alternative approach is for the PL to issue an interrupt once the computation is finished, thus triggering an interrupt routine of the PS. This approach is far more efficient, as the PS can run other tasks until the interrupt is received, while the PS can handle the inferred label instantly. Therefore, *interrupts* are chosen.

4.4.2 Comminucation Procedure [Markus]

This section gives an overview of how the PS uses the LeNet-5 hardware accelerator to efficiently classify a handwritten digit and retrieve the corresponding label. As explained in Section 4.4.1, for this, DMA and interrupts are used. Figure 18 illustrates this simplified procedure.
First, the PS stores the image of a handwritten digit to be classified in RAM and then sends the corresponding address to the DMA controller. Additionally, the

PS instructs the DMA controller to store the resulting label in another dedicated memory location. After that, the DMA controller initiates the data transfer and redirects the image stream to the hardware accelerator, which starts inferring the image. This does not require further intervention of the PS. Once the inference is finished, the hardware accelerator sends the corresponding image label to the DMA controller, which stores it in the previously specified memory location. After that, the DMA controller issues an interrupt to the PS and thus triggers the interrupt handler of the PS, which now can retrieve the inferred label from RAM and use it for further processing.
This procedure shows the advantage of DMA and interrupts as the PS can run other tasks while the inference runs and is instantly notified once the image label is available.

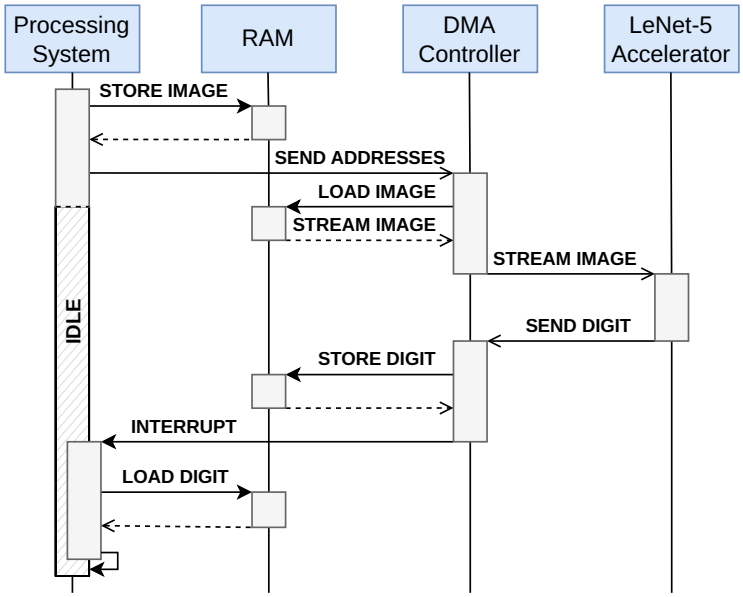


Fig. 18: The Communication Procedure between the PS and the Hardware Accelerator that makes use of DMA and Interrupts

4.4.3 Memory Interconnect [Markus]

For communicating with the PS, the memory interconnect on the PL uses the Advanced eXtensible Interface (AXI) bus protocol to send and receive data via the system's main bus. Additionally, DMA and interrupts are used, as outlined in Section 4.4.1.

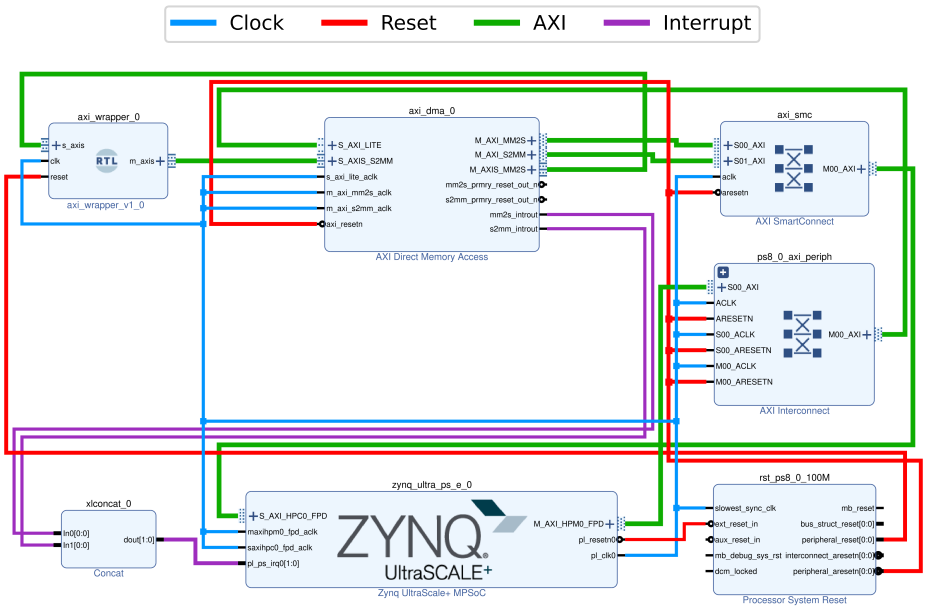


Fig. 19: Overview of the whole Memory Interconnect of the LeNet-5 Accelerator

Figure 19 shows an overview of the whole memory interconnect of the LeNet-5 accelerator implemented as a Vivado Block Design. In it, the main *Zynq UltraScale+ MPSoC Processing System* block is connected to an *AXI-Interconnect* and an *AXI-SmartConnect* block, which both are connected to the *AXI-DMA* block. This *AXI-DMA* block handles all DMA transactions and converts them into a simple AXI protocol. In this case, this consists of a `tdata`, `tkeep`, `tlast`, `tvalid`, and a `tready` signal.
The *AXI-Wrapper* block, which manages the hardware accelerator, is connected with two sets of these signals. With one set, the *AXI-Wrapper* receives data (*slave*), and with the other set, it transmits data (*master*). Both of these connections use an 8-bit data width, which is the maximum amount of data the LeNet-5 accelerator can process per clock cycle. For this reason, *tkeep* is neglected in the following.
Figure 20 shows the simplified state chart of the *AXI-Wrapper*, which only shows state transitions that result in a state change. The AXI signals that correspond

to receiving data are labeled `s_[SIGNAL]`. The signals that correspond to transmitting data are labeled `m_[SIGNAL]`.
The *AXI-Wrapper* has three primary states: *Receive image*, *Wait for Result*, and *Send Digit & Reset Accelerator*. In the first state, the *AXI-Wrapper* receives the whole image from the PS and forwards it to the hardware accelerator, which already starts inference when receiving the first pixel. After the whole image (1 KiB) has been received, the *AXI-Wrapper* signals that it cannot receive data anymore (`s_tready = 0`) and waits for the hardware accelerator to finish inference. Once the hardware accelerator has finished inference, it sends the inferred label to the *AXI-Wrapper*, which forwards it to the PS and resets the hardware accelerator for the subsequent inference. After the label has been transmitted successfully, the *AXI-Wrapper* is ready to receive the next image and, therefore, transitions back to the *Receive image* state.

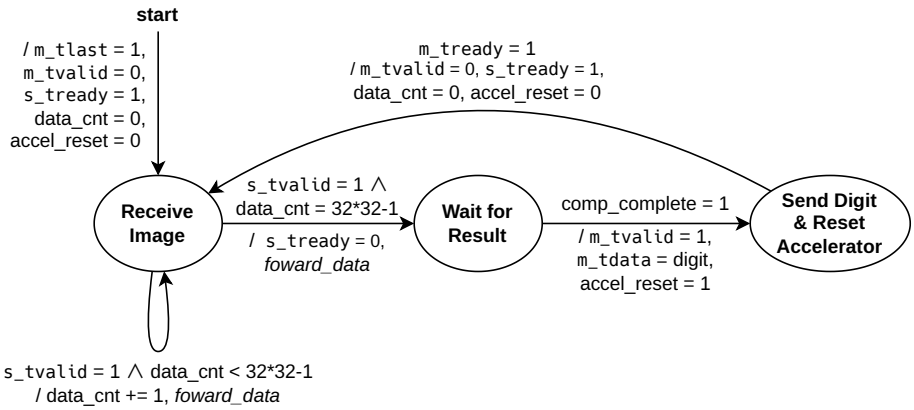


Fig. 20: State Chart visualizing the different States of the AXI-Wrapper that manages the LeNet-5 Hardware Accelerator

To verify the components introduced in this section independently of the hardware accelerator, a simple accumulator can be employed instead of the LeNet-5 accelerator. This accumulator simply adds all incoming bytes and outputs the result once no data is received anymore. This procedure is similar to the one for the LeNet-5 hardware accelerator and, therefore, helps to ensure that all components are set up correctly. For implementation details, please refer to the Vivado project stored in *SW-Integration/Vivado*.

5 Evaluation [Shared]

Different aspects are considered when evaluating the LeNet-5 hardware accelerator. First, Section 5.1 analyzes the resource consumption of the implemented design on the target hardware, namely the Ultra96-v2 evaluation board. It also shows the impact of different bit widths. Second, Section 5.2 benchmarks the LeNet-5 hardware accelerator and compares its latency to various baselines.

5.1 Resource Consumption [Maxim]

In order to evaluate the scalability of our design, let us first talk about the resource consumption on our target platform, the Ultra96-V2 board (Section 1).

Site Type	Used	Available	Util%
CLB LUTs	51274	70560	72.67
LUT as Logic	49761	70560	70.52
LUT as Memory	1513	28800	5.25
CLB Registers	36683	141120	25.99
Register as Flip Flop	36683	141120	25.99
Register as Latch	0	141120	0.00
Block Ram Tile	82.5	216	38.19
DSPs	0	360	0.00
Carry8	4721	8820	53.53
F7 Muxes	421	35280	1.19
F8 Muxes	168	17640	0.95
F9 Muxes	0	8820	0.00

Table 1: Resource Utilization after Synthesis of Top-Level Module *lenet_accel* on the Ultra96-V2 Platform

Table 1 shows the Resource utilization on the Ultra96-v2 board after the Synthesis step. The first thing apparent from this table is the high LUT utilization, using $\approx \frac{3}{4}$ of the system's overall space. From this alone, we can safely say that if our design significantly increased in complexity, it would likely not fit onto the board. The Memory usage, on the other hand, is far below what could be considered to be critical, with only $\approx \frac{1}{4}$ of all registers and $\approx \frac{1}{3}$ of all BRAM tiles being used. DSPs are not used because of our overall bit width of $vector_length = 8$. From this table, it is safe to say that our design is mostly LUT bottlenecked, which is further confirmed in Figure 21. It illustrates the normalized resource utilization of LUTs, FFs, and BRAM tiles for $vector_length \in \{1, \ldots, 16\}$.

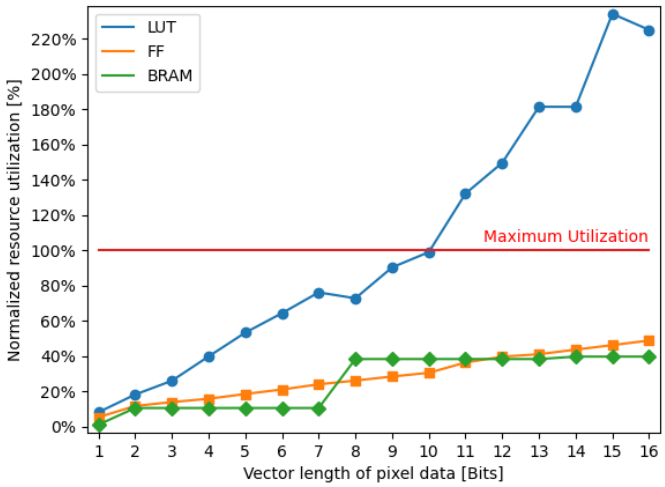


Fig. 21: Normalized Resource Utilization after Synthesis of Top-Level Module *lenet_accel* for different *vector_length* on the Ultra96-v2 Board (no DSPs)

Throughout all benchmarks shown in the figure, no DSP slices were utilized to have an adequate baseline for comparison because, for bit widths such as 8 Bit or lower, no DSPs are utilized by the synthesis tool.
The figure illustrates how the design features a relatively gradual and linear increase in memory consumption when looking at the FlipFlop usage and no significant increase for the inferred BRAM blocks. On the other hand, the LUT count increases rapidly and non-linearly, which shows us that realistically, only 9 bits could still be achieved for the implementation. For bit widths larger than that, DSP slices or a different platform altogether would be necessary.
There are also a few other more nuanced details, such as that even bit widths, especially bases of two, feature a much slower increase or less LUT consumption when looking at bases of two. This behavior probably has to do with how BRAM tiles and their addressing logic are inferred, but more investigation would be needed to confirm or deny this claim.

5.2 Performance Benchmarking [Markus]

5.2.1 Setup [Markus]

For evaluating the performance of the LeNet-5 hardware accelerator, it is benchmarked and compared against different baselines. The baselines consist of the following: (1) Brevitas with the PyTorch backend running on the CPU of a desktop system, namely an *AMD Ryzen 5 5500U*, (2) the LeNet-5 software implementation running on the same previously mentioned desktop system, and (3) the LeNet-5 software implementation running exclusively on the PS of the

Ultra96-v2 evaluation board. For each of these baselines, the batch size is set to one. In the following, an overview of the setup for each corresponding benchmark is given:

(1) Brevitas (+ PyTorch Backend) [AMD Ryzen 5 5500U]. For this baseline, inference is executed via a Python script using the Brevitas library with the PyTorch backend having batch size set to one. The script runs on a laptop containing an *AMD Ryzen 5 5500U* CPU[3] and DDR4 memory running at 3200 MHz. Latency is measured via the global system time.

(2) Software Implementation [AMD Ryzen 5 5500U]. This implementation uses the C++ software implementation and runs it on the same machine as the previous baseline. The software implementation is configured to run with a batch size of one and clipped 8, and 24 bit integers as the main and accumulator data types, respectively. Test images and layers are parsed from the *Numpy* and *ONNX* format, respectively, and are preprocessed before starting benchmarking. The latency is measured via the global system time.

(3) Software Implementation [Avnet Ultra96-v2]. Instead of the previously mentioned desktop system, the software implementation is benchmarked as a bare metal application on the PS of the Ultra96-v2 evaluation board. More specifically, this benchmark is executed on its *ARM Cortex-A53* CPU[4] with LPDDR4 memory running at 533 MHz. Not all libraries are supported on this embedded hardware. Test images and layers are parsed and preprocessed on a desktop system, then serialized and directly linked into the application's binary. When starting the application, the PS deserializes the data and then runs the inference for every image. The latency is measured via the physical timer counter register.

(4) Hardware accelerator [Avnet Ultra96-v2]. For this benchmark, the hardware accelerator runs on the PL of the Ultra96-v2 evaluation board with a frequency of 250 MHz and is integrated as explained in Section 4.4. Similarly to the previous implementation, the test images are linked into the application's binary to be easily used at runtime. As the batching size is set to one, the application sends one image to the LeNet-5 hardware accelerator and measures the time until the result is available. The system is notified via an interrupt. See Section 4.4.2 for more information on how communication works.

5.2.2 Results [Markus]

Figure 22 shows **(a)** the latency of all previously introduced LeNet-5 implementations, and **(b)** the speedup that the hardware accelerator achieves compared to the different baselines. Both figures show that the hardware accelerator clearly outperforms all baselines in terms of latency by a huge margin. Not only does the

[3] The *AMD Ryzen 5 5500U* CPU has the following specifications: 6 cores, 12 threads, a base frequency of 2.1 GHz, and a turbo frequency of 4.0 GHz

[4] The *ARM Cortex-A53* CPU has the following specifications: 4 cores, 4 threads, and a clock frequency of 1.8 GHz

hardware accelerator outperform both implementations running on the desktop system (namely Brevitas and the software implementation), but it also provides a speedup of *711x* for the software implementation running on the PS of the Ultra96-v2 evaluation board. This is a reduction regarding latency of *99.86%*. This benchmark shows that the hardware accelerator as specialized hardware for the LeNet-5 architecture is capable of providing a significant speedup compared to software implementations running on general-purpose hardware.

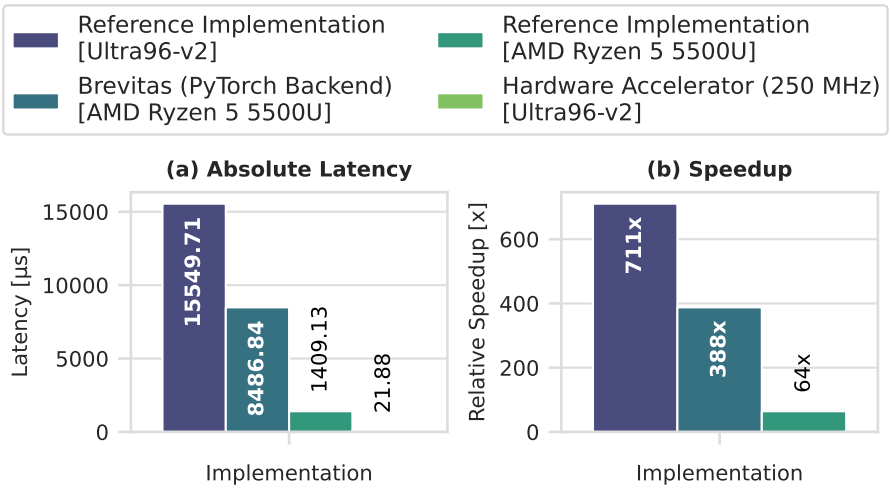


Fig. 22: Comparison of the Latency between the LeNet-5 Hardware Accelerator and different Baselines

6 Future Work [Shared]

There exist several possibilities for future work regarding the LeNet-5 hardware accelerator. Potential performance optimizations are discussed in Section 6.1. Section 6.2 gives an overview of how to further reduce the required hardware and in Section 6.3, additional features are proposed.

6.1 Performance Optimizations [Shared]

Queueing. The approach to send one image and wait until inference is finished before sending the next image, as used in Section 5.2, seems suboptimal. With this approach, approximately *15%* of the inference time is spent until the first byte of the pixel stream arrives at the hardware accelerator. Alternatively, a queue could be implemented right before the hardware accelerator, buffering incoming images and starting the inference for the next image as soon as the previous one is finished. This approach will increase throughput but will not improve latency.

Pipelining. Currently, the hardware accelerator is not fully pipelined. That is to say, the hardware accelerator waits until the whole image is processed before starting the following image. Additional work could fully pipeline the hardware accelerator to increase throughput.

Optimize Critical Paths. Future work could also optimize critical paths in the design to increase the maximum clock frequency, further decreasing latency and increasing throughput.

6.2 Hardware Optimizations [Shared]

Currently, the hardware accelerator is implemented to use 8-bit and 24-bit as the main and accumulator types, respectively. However, as Section 2.2.2 pointed out, reducing the accumulator to only 16 bits seems possible but would need additional evaluation to ensure this works for other images.

6.3 Additional Features [Shared]

The current implementation of the LeNet-5 hardware accelerator fixes all weights and kernels during synthesis. Therefore, it is not possible to adapt to new situations (e.g., to new weather conditions for outdoor applications) or to fine-tune the model during runtime. To support this, future work could introduce a new DMA channel that allows the PS to send new parameters to the hardware accelerator to update the BRAM blocks during runtime. To avoid race conditions, the new parameters could be buffered until the current inference is finished.

7 Conclusion [Shared]

This report presented the development of a customized LeNet-5 hardware accelerator targeted for FPGAs, specifically the Ultra96-v2 evaluation board, aiming for applications demanding high efficiency and performance. As prerequisites, quantization-aware training was employed to reduce the hardware requirements while maintaining high accuracy, and a software reference was designed and implemented as a key tool to simplify and assist the hardware development process. After this, the design and implementation of the accelerator were detailed, emphasizing its high optimization, parallelism, and adaptability to other CNN architectures using a fully generic design. Furthermore, to integrate the accelerator with the PS, a highly performant approach with DMA and interrupts is used. Finally, the architecture was evaluated and benchmarked by comparing it to different baselines. This showed that the highly-specialized hardware accelerator developed clearly outperforms all baselines regarding latency and reduces latency by as high as *99.86%*. Therefore, the developed hardware accelerator accomplishes its goal of providing a highly efficient and high-performing way to run inference for the LeNet-5 architecture.

References

1. Achieving FP32 Accuracy for INT8. Accessed: 2025-02-28. (Cited on 3)
2. Arg max – Wikipedia. Accessed: 2025-03-22. (Cited on 18)
3. Graphic of LeNet-5 Architecture. Accessed: 2025-02-26. (Cited on 2)
4. Libnpy at 471fe480. Accessed: 2025-04-25. (Cited on 6)
5. Documentation of ONNX. Accessed: 2025-02-28. (Cited on 3)
6. Protocol Buffers Documentation. Accessed: 2025-03-15. (Cited on 6)
7. Documentation of PyTorch. Accessed: 2025-02-28. (Cited on 3)
8. Product specification of the Ultra96 board. Accessed: 2025-04-07. (Cited on 2)
9. Documentation of Brevitas. Accessed: 2025-02-28. (Cited on 3)
10. Luis Aranda, Pedro Reviriego, and Juan Antonio Maestro. Protecting image processing pipelines against configuration memory errors in SRAM based FPGAs. *Electronics*, 7:322, 11 2018. (Cited on 11)
11. Yann Lecun, Leon Bottou, Y. Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86:2278 – 2324, 12 1998. (Cited on 2)
12. Runbin Shi, Justin S.J. Wong, and Hayden K.-H. So. High-throughput line buffer microarchitecture for arbitrary sized streaming image processing. *Journal of Imaging*, 5(3), 2019. (Cited on 12)
13. Quang Hieu Vo, Ngoc Linh Le, Faaiz Asim, Lok-Won Kim, and Choong Seon Hong. A deep learning accelerator based on a streaming architecture for binary neural networks. *IEEE Access*, 10:21141–21159, 2022. (Cited on 1, 2)